

CS-DNO: a Craig–Sulem-Structured Neural Dirichlet–Neumann Operator

Architecture reference for `models/dno-net/dno_net_v2.py` (baseline run `cs_dno_w512b8_l256_v3_20260605_032608`)

June 11, 2026

1 What the model replaces, and where it sits

The water-wave system is evolved in Zakharov form: with $\eta(x, t)$ the free surface and $\xi(x, t)$ the surface trace of the velocity potential,

$$\eta_t = \mathcal{G}(\eta; h) \xi, \quad (1)$$

$$\xi_t = -g\eta - \frac{1}{2}\xi_x^2 + \frac{(\mathcal{G}(\eta; h)\xi + \eta_x\xi_x)^2}{2(1 + \eta_x^2)}, \quad (2)$$

where $\mathcal{G}(\eta; h)$ is the *Dirichlet–Neumann operator* (DNO): solve Laplace’s equation in the fluid domain $\{-h < y < \eta(x)\}$ with Dirichlet data ξ on the surface and no-flux at the flat bottom, and return the (scaled) normal derivative of the potential at the surface. Every right-hand-side evaluation requires one application of $\mathcal{G}$; the reference implementation is the order-6 Craig–Sulem series with $8\times$ zero-padding in f64 (`dno_series_eval`), which dominates the cost of a rollout. CS-DNO is a neural surrogate for the map

$$(\eta, \xi, h) \longmapsto \mathcal{G}(\eta; h) \xi$$

on the $n_x = 1024$ periodic grid ($L = 2\pi$), trained on snapshots harvested from f64 solver trajectories. At rollout time it is dropped into the same time integrator in place of the series evaluation (`solver/evals/model_rollout.py`). One convention matters everywhere: the entire data pipeline — solver RHS, post-step states, and stored labels — lives in the dealiased subspace $k \leq 0.25 k_{\max}$ (cutoff $k = 128$), because the order-6 series amplifies high- k roundoff like k^6 . The surrogate therefore only ever sees and is only ever scored on that subspace.

2 Mathematical structure the architecture encodes

The DNO admits the Craig–Sulem expansion

$$\mathcal{G}(\eta; h) = \sum_{j \geq 0} \mathcal{G}_j(\eta; h), \quad \mathcal{G}_j \text{ homogeneous of degree } j \text{ in } \eta,$$

whose first terms (finite depth, $D = -i\partial_x$, $|D| = \text{multiplier } |k|$) are

$$\mathcal{G}_0(h) = |D| \tanh(h|D|), \quad (3)$$

$$\mathcal{G}_1(\eta; h) \xi = -\mathcal{G}_0(\eta \mathcal{G}_0 \xi) - \partial_x(\eta \partial_x \xi). \quad (4)$$

Each $\mathcal{G}_j$ is built by *alternating Fourier multipliers* $m(D, h)$ with *pointwise multiplications by powers of η* . Four structural properties of the true operator drive the design:

- P1 Linearity in ξ .** $\mathcal{G}(\eta; h)$ is a linear operator applied to ξ ; all nonlinearity enters through η .
- P2 Analyticity in η .** The expansion converges for moderate slopes; the η -dependence is polynomial order by order.
- P3 Known dispersion.** The linear part $\mathcal{G}_0 = |D| \tanh(h|D|)$ is exact and closed-form, with all depth dependence through $\tanh(hk)$.
- P4 Multiplier–multiplication factorization.** Higher corrections look like $m_{\text{out}}(D, h) [\phi(\eta) \cdot m_{\text{in}}(D, h) \xi]$ composed and summed.

CS-DNO hard-codes P1, P3 and P4 and soft-codes P2. By contrast, a generic FNO fed (η, ξ) as channels respects none of them: it can (and measurably does) produce output nonlinear in ξ , which on near-linear states materializes as spurious sideband energy; and it must spend capacity re-learning $\tanh(hk)$ from data. The predecessor **SpectralDNO** already enforced P1 via a single rank- r “spectral sandwich” $C(\eta)^\top F(\eta) C(\eta)$ added to $\mathcal{G}_0$, but its η -trunk was an FNO-style stack of spectral convolutions and its per-state filter F was produced by a global pooled MLP — depth entered only as an input channel. CS-DNO replaces that with a sum of explicitly Craig–Sulem-shaped blocks and analytic depth conditioning.

3 Architecture

3.1 Interface and normalization

The module signature is `(inputs, depth) → gxi` with `inputs` of shape $(B, 1024, 2)$ holding normalized $(\hat{\eta}, \hat{\xi})$ and `depth` of shape $(B, 1)$ holding $\log h$. Under `norm=scale` each quantity is divided by its training-corpus absolute maximum:

$$\hat{\eta} = \eta / s_\eta, \quad \hat{\xi} = \xi / s_\xi, \quad \widehat{\mathcal{G}\xi} = \mathcal{G}\xi / s_g,$$

with $s_\xi = 0.14096$ and $s_g = 0.76948$ for the v3 corpus (recorded in `config.json`; recomputed per dataset). Internally the model un-scales ξ to physical units before every operator application and re-scales the output, so the learned parts operate on physically meaningful fields. Depth is clipped as $\log h \mapsto \min(\log h, \log 5)$: for $k \geq 1$, $\tanh(5k)$ is within 10^{-4} of 1, so all depths beyond $h = 5$ are physically indistinguishable from infinite depth and the clip simply saturates that direction of input space.

3.2 Exact linear baseline

The flat-surface DNO is applied analytically in Fourier space,

$$\mathcal{G}_0(h) \xi = \text{irfft} [k \tanh(hk) \cdot \text{rfft}(\xi)],$$

and the network only ever learns the *residual* $\mathcal{G} - \mathcal{G}_0$. Because every learned contribution is zero-initialized (below), the model output at step 0 of training is exactly $\mathcal{G}_0 \xi$ — training starts from the linear water-wave theory rather than from noise.

3.3 The η feature stack

The surface shape is expanded into seven fixed analytic channels — pointwise powers plus spectral derivatives:

$$[\eta, \eta^2, \eta^3, \partial_x \eta, \partial_x^2 \eta, |D|^{1/2} \eta, \mathcal{H} \eta],$$

where $\mathcal{H}$ is the Hilbert transform ($-i \text{sgn}(k)$) and the half-derivative $|D|^{1/2}$ is natural in deep-water $\mathcal{G}_1$ scalings. The powers give the polynomial orders of P2 directly; the derivatives expose slope and

curvature (the physical small parameters of the expansion); $\mathcal{H}$ and $|\mathbf{D}|^{1/2}$ give the trunk access to odd/phase-shifted and fractional information about η that pointwise powers cannot produce. These seven channels pass through a small pointwise trunk shared by all blocks,

$$\text{Dense}(7 \rightarrow 256) \rightarrow \text{gelu} \rightarrow \text{Dense}(256 \rightarrow 256) \rightarrow \text{gelu},$$

producing η -features $E(\eta) \in \mathbb{R}^{B \times 1024 \times 256}$ (width/2 channels). Note the trunk is *pointwise in x* apart from the fixed spectral derivatives — there are no learned convolutions over x anywhere in the model; all learned spatial coupling happens through Fourier multipliers.

3.4 Depth-aware Fourier multipliers

Every learned multiplier is produced by `DepthAwareMultiplier`: a 2-layer MLP evaluated *per mode* on physical dispersion features,

$$m(k, h) = W_2 \tanh(W_1 [k, h, \tanh(hk), k \tanh(hk)] + b_1) + b_2 \in \mathbb{R}^{B \times 513 \times r},$$

with hidden width 128 and r output channels (one per branch). Because the MLP sees $\tanh(hk)$ and $k \tanh(hk)$ as inputs, the exact dispersion scaling — and in particular the correct shallow/deep crossover in kh — is available as a *linear readout* rather than something to be discovered; the network learns perturbations on physics, not the physics itself. This is the entire mechanism of depth conditioning: h never enters as a spatial channel, only through these multipliers (and the $\mathcal{G}_0$ baseline). The multipliers are real-valued, i.e. zero-phase filters (a representational choice discussed in §3.6).

3.5 The Craig–Sulem block

Each of the $n_{\text{blocks}} = 8$ blocks computes a sum of $r = 256$ branches shaped exactly like a Craig–Sulem term (P4):

$$\text{Block}(\eta, \xi, h) = \sum_{i=1}^r m_{\text{out}}^{(i)}(\mathbf{D}, h) \left[\underbrace{\Phi_i(\eta)}_{\text{pointwise in } x} \cdot (m_{\xi}^{(i)}(\mathbf{D}, h) \xi) \right],$$

implemented as:

1. $\Phi(\eta) = \text{Dense}(256 \rightarrow r) E(\eta)$ — a **zero-initialized** linear projection of the shared trunk features to one spatial gain field per branch (this is what makes the whole block inert at init);
2. ξ -branching in k -space: $\hat{\xi}_i = m_{\xi}^{(i)}(k, h) \hat{\xi}(k)$, one filtered copy of the *physical* ξ per branch;
3. a *pointwise* product $\Phi_i(\eta)(x) \cdot \xi_i(x)$ — deliberately diagonal over branches (no outer product between η -channels and ξ -branches), so cost and memory are linear in r ;
4. per-branch output multipliers $m_{\text{out}}^{(i)}$, then a sum over branches.

The full model is the baseline plus the block sum, rescaled to normalized units:

$$\mathcal{G}_{\theta}(\eta; h) \xi = \mathcal{G}_0(h) \xi + \sum_{b=1}^8 \text{Block}_b(E(\eta), \xi, h)$$

Linearity in ξ (P1) holds *by construction*: ξ enters every branch exactly once, through a linear filter, a multiplication by an η -dependent field, and another linear filter. For fixed (η, h) the model *is* a linear operator in ξ — a sum of $8 \times 256 = 2048$ multiplier–gain–multiplier sandwiches. Each block can represent $\mathcal{G}_1$ -shaped terms exactly when Φ recovers an affine function of η , and higher-order $\mathcal{G}_j$ effectively, since the trunk makes $\Phi_i(\eta)$ an arbitrary (learned, pointwise) nonlinear functional of the seven η -channels.

3.6 Expressivity notes and deliberate trade-offs

- **Zero-phase multipliers.** Real $m(k, h)$ on rfft modes are even, zero-phase convolution kernels. Odd differential action on ξ is therefore not exactly representable: of the analytic $\mathcal{G}_1 = -\mathcal{G}_0 \eta \mathcal{G}_0 - \partial_x \eta \partial_x$, the first term and the $\eta \partial_x^2$ part of the second are exactly reachable ($\mathcal{G}_0$ and k^2 are real multipliers, with $\eta, \partial_x^2 \eta$ available to Φ), while the $\eta_x \partial_x$ part can only be approximated through combinations (the η -side does have odd operators via $\partial_x \eta$ and $\mathcal{H}\eta$). This is a deliberate economy — complex multipliers double the parameter count of the dominant tensors — and at the achieved single-step error ($\sim 10^{-3}$ relative H^1) it has not been the binding constraint.
- **No enforced self-adjointness.** The true $\mathcal{G}$ is self-adjoint and positive semi-definite; the baseline run learns $m_\xi \neq m_{\text{out}}$ independently, so symmetry is only approximate. A `tie_xi_out_mult` flag exists that sets $m_\xi = m_{\text{out}}$ per branch, making each branch manifestly self-adjoint (see §8).
- **Exact flat-surface limit only at init.** With biases present, $\Phi(0) \neq 0$ after training, so $\mathcal{G}_\theta(0; h) \neq \mathcal{G}_0(h)$ exactly (it is close in practice). A `phi_bias_free` flag removes all trunk/projection biases so the residual vanishes identically at $\eta = 0$ for *all* weights (§8).
- **Global support in x .** Although Φ is pointwise, the multipliers are dense in k , so the learned operator has full nonlocal reach — consistent with the true DNO, which is nonlocal.

4 Hyperparameters and parameter budget

field	value	meaning
<code>width</code>	512	trunk feature width is <code>width</code> /2 = 256
<code>n_blocks</code>	8	Craig–Sulem blocks summed
<code>latent</code>	256	branches r per block
<code>cs.n_polys</code>	3	η, η^2, η^3 features
<code>cs.use*_deriv / hilbert</code>	all true	$\partial_x, \partial_x^2, D ^{1/2}, \mathcal{H}$ features
<code>cs.mult_hidden</code>	128	multiplier MLP hidden width
<code>h_clip_max</code>	5.0	depth saturation
<code>modes</code>	64	<i>unused</i> (CLI compatibility with FNO/SpectralDNO)

Table 1: Baseline architecture hyperparameters (`cs_dno_w512b8_l256_v3_20260605_032608`).

component	params	share
η trunk (<code>eta_feat_proj</code> 7→256 + <code>eta_feat_mix</code> 256→256)	67,840	6.0%
Φ projections ($8 \times$ <code>Dense</code> 256→256, zero-init)	526,336	46.5%
multiplier MLPs ($8 \text{ blocks} \times \{m_\xi, m_{\text{out}}\} \times (4 \rightarrow 128 \rightarrow 256)$)	538,624	47.6%
total	1,132,800	

Table 2: Exact parameter budget (matches `param_count` in the run config). For comparison, the FNO baseline `fno_w128b6_v3_hclip5` has 13.1M parameters.

The model is small because it learns only scalar gain fields and 1-D multiplier curves on top of hard-coded structure — there are no $512 \times 512 \times 64$ complex spectral weight tensors anywhere.

5 Training protocol

Data. `combined_dataset_v3.npz`: 5,993,989 snapshots $(\eta, \xi, \mathcal{G}\xi, h)$ at $n_x = 1024$, harvested from f64 solver rollouts across ten families — Tanaka solitons g0/g1 (2.0M), Benjamin–Feir g0/g1/modal (2.03M), Stokes deep/finite (1.0M), linear shallow sweeps (0.5M), random seas deep/finite (0.46M). Labels are the order-6/pad-8 operator *lowpassed to the $k \leq 128$ subspace* (the pipeline’s invariant subspace). Split 80/10/10 by a seeded global permutation: 4,795,193 train / 599,398 val. The v4 corpus in the current retrain appends a shallow–steep wavetrain family (+480k, source 11) and changes nothing else; see `v4_dataset_report.pdf`.

Loss. Relative Sobolev H^1 in the spectral domain, on normalized targets:

$$\mathcal{L} = \text{mean}_{\text{batch}} \frac{\|w \odot (\widehat{\text{pred}} - \widehat{\text{tgt}})\|_2}{\|w \odot \widehat{\text{tgt}}\|_2}, \quad w(k) = \sqrt{1 + k^2},$$

i.e. an L^2 -plus-gradient norm that up-weights exactly the high- k content a rollout differentiates; relative (per-sample) so soliton rows and near-flat rows contribute comparably.

Optimization. AdamW, base lr 2×10^{-4} with single cosine decay over the full run, weight decay 10^{-4} , global batch 256, 40 epochs, f32 training throughout. The baseline trained on one GPU; the trainer shards the global batch across devices via `shard_map` with per-step gradient all-reduce (the v4 retrain uses both GPUs this way — optimization-identical to single-device). Best-val checkpoint is kept: **val** = 9.28×10^{-4} relative H^1 , vs 3.69×10^{-3} for the 13.1M-parameter FNO baseline on the same data and loss (80 epochs) — $4\times$ lower error with $11.6\times$ fewer parameters at half the epochs.

6 Deployment in rollouts

`model_rollout.py` reconstructs the module from `config.json` and substitutes it for the series evaluation inside the same time integrator used to generate truth, with the same 0.25 dealias filter applied to the RHS and post-step states — the surrogate rollout lives in the same invariant subspace it was trained on. The network always evaluates in f32; with `--f64_harness` (the honest configuration — see `v4_dataset_report.pdf`) all surrounding integrator state and spectral algebra run in f64. Depth enters the surrogate as $\log h$ exactly as in training.

7 Empirical status (baseline, f64 harness, truth-gated)

regime	divergence	median rel- $L^2(\eta)$ at $t=200$
bf_g0	0%	0.0044
bf_g1	0%	0.0067
bf_modal	6.7%	0.0055
tanaka_g0	6.2%	0.0420
tanaka_g1	18.8%	0.0443
linear (shallow-steep sweep)	27%	0.084

Table 3: Long-rollout accuracy of the v3 baseline. The single systematic weakness is shallow water ($h \lesssim 0.1$) at moderate steepness ($a/h \gtrsim 0.15$) on non-soliton states — a training-coverage gap, not an architecture failure (single-step error on exact solitons is $\sim 2 \times 10^{-4}$ even where rollouts die). The v4 retrain targets exactly this with the shallow–steep family.

8 Parked architectural variants

Three flags exist in the module, are plumbed through training and rollout, and are **off** in the baseline; the program that motivated them was closed when the f32 FFT noise floor was identified as the real obstacle:

- **use_g1_baseline** — add the closed-form $\mathcal{G}_1(\eta; h)\xi = -\mathcal{G}_0(\eta \mathcal{G}_0\xi) - \partial_x(\eta \partial_x\xi)$ to the analytic baseline so learned blocks model only $\mathcal{G}_{2+}$. Requires **g1_k_cut** (default 128): the doubly k -amplified f32 FFT chain produces rounding noise above $k \approx 128$ that exceeds the genuine $\mathcal{G}_1$ signal there, which the H^1 loss then up-weights into an unlearnable $\sim 3 \times 10^{-3}$ floor — the masked variant zeroes the term’s output modes at $k \geq \text{cut}$.
- **tie_xi_out_mult** — share one multiplier as both m_ξ and m_{out} per branch, making every branch (hence every block) self-adjoint in ξ , matching the true operator’s symmetry.
- **phi_bias_free** — remove all biases from the η trunk and Φ projections so every learned term vanishes identically at $\eta = 0$: the flat-surface limit $\mathcal{G}_\theta(0; h) = \mathcal{G}_0(h)$ becomes exact for all trained weights, not just at init.